目录

基本模版 1

一般图最大权匹配 1

一般图最大独立集 5

全图割 6

基本模版

```
#define gep(k,from,Head,Er) for(int k=Head[from];k!=-1;k=Er[k].nxt)
#define rep(i,a,b) for(int i=a,rep##i=b;i<=rep##i;i++)
#define per(i,a,b) for(int i=a,rep##i=b;i>=rep##i;i--)
#define sz(x) (static_cast<int>(x.size()))
#define all(x) x.begin(),x.end()
template<typename T>void Clear(T&x){T y;x.swap(y);}
```

一般图最大权匹配

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll Inf = (ll)1e18;
const int MaxV = 410; // 可按需要调整（>= 实际顶点数）
struct Edges {
    int from, to;
    int data;
    Edges(int _from = 0, int _to = 0, int _data = 0) : from(_from), to(_to), data(_data) {}
};
int sizev; // 原图顶点数（不含“花”的扩展节点）
int tot;   // 当前编号的最大节点（包括扩展出来的花节点）
vector<vector<Edges>> Eij; // Eij[i][j] 保存 i->j 的边信息（用矩阵语义便于花合并时更新）
vector<vector<int>> Root;  // Root[flower][j] = 哪个原点在花内连接到 j（用于花内边的代表选择）
vector<vector<int>> Leaf;  // Leaf[f] : 花 f 的成员；若 f<=sizev, Leaf[f] 包含一个元素 f（约定）
vector<int> Match, Pre, Mark, Dad, Visit, Slack;
vector<ll> Expect;
queue<int> Q;
int GetLca(int x, int y) {
    static int tim = 0;
    if (++tim == INT_MAX) { tim = 1; fill(Visit.begin(), Visit.end(), 0); }
    while (x != 0) {
        Visit[x] = tim;
        x = Dad[Match[x]];
        if (x != 0) x = Dad[Pre[x]];
    }
    while (y != 0) {
        if (Visit[y] == tim) return y;
        y = Dad[Match[y]];
```

```
        if (y != 0) y = Dad[Pre[y]];
    }
    return 0;
}
inline ll CalcEdge(const Edges &e) { // 边代价 / slack 的计算（依据 Expect 顶标）
    return Expect[e.from] + Expect[e.to] - (ll)Eij[e.from][e.to].data * 2;
}
void UpdateSlack(int id, int now) { // 更新 now 对应的 Slack（记录能使 now 匹配的代表点 id）
    if (!Slack[now] || CalcEdge(Eij[id][now]) < CalcEdge(Eij[Slack[now]][now])) Slack[now] = id;
}
void CalcSlack(int now) {
    Slack[now] = 0;
    for (int i = 1; i <= sizev; ++i)
        if (Eij[i][now].data > 0 && Dad[i] != now && Mark[Dad[i]] == 0)
            UpdateSlack(i, now);
}
void JoinNodeToQueue(int now) {
    if (now <= sizev) { Q.push(now); return; }
    for (int v : Leaf[now]) JoinNodeToQueue(v);
}
void SetDadRec(int now, int dad) {
    Dad[now] = dad;
    if (now <= sizev) return;
    for (int v : Leaf[now]) SetDadRec(v, dad);
}
void SetMatch(int from, int to) { // 为 from 设置匹配到 to（处理花的展开/交错路径切换）
    Match[from] = Eij[from][to].to; // 保证 Eij[from][to].to 存储花/点对应“匹配的点”
    if (from <= sizev) return;
    int root = Root[from][Eij[from][to].from];
    auto it = find(Leaf[from].begin(), Leaf[from].end(), root);
    int place = (int)(it - Leaf[from].begin());
    if (place % 2 == 1) { reverse(Leaf[from].begin() + 1, Leaf[from].end()); place = (int)Leaf[from].size()
    for (int i = 0; i < place; ++i) SetMatch(Leaf[from][i], Leaf[from][i ^ 1]);
    SetMatch(root, to);
    rotate(Leaf[from].begin(), Leaf[from].begin() + place, Leaf[from].end());
}
void BlossomBreak(int now) { // 当一个花“无欲望”时，把其中不应保留的子花炸开
    for (int v : Leaf[now]) {
        if (v > sizev && !Expect[v]) BlossomBreak(v);
        else SetDadRec(v, v);
    }
    Dad[now] = 0;
}
void BlossomBuild(int from, int lca, int to) {
    int now = sizev + 1;
    while (now <= tot && Dad[now]) ++now;
    if (now > tot) ++tot;
    Expect[now] = Mark[now] = 0;
    Match[now] = Match[lca];
    Leaf[now].clear();
    Leaf[now].push_back(lca);
    for (int k = from; k != lca; k = Dad[Pre[ Dad[ Match[k] ] ] ]) {
        Leaf[now].push_back(k); Leaf[now].push_back(Dad[Match[k]]);
```

```
        JoinNodeToQueue(Dad[Match[k]]);
    }
    reverse(Leaf[now].begin() + 1, Leaf[now].end());
    for (int k = to; k != lca; k = Dad[Pre[ Dad[ Match[k] ] ] ]) {
        Leaf[now].push_back(k); Leaf[now].push_back(Dad[Match[k]]);
        JoinNodeToQueue(Dad[Match[k]]);
    }
    SetDadRec(now, now);
    for (int i = 1; i <= tot; ++i) { Eij[now][i].data = Eij[i][now].data = 0; Root[now][i] = 0; }
    for (int vex : Leaf[now]) {
        for (int j = 1; j <= tot; ++j)
            if (!Eij[now][j].data || CalcEdge(Eij[vex][j]) < CalcEdge(Eij[now][j]))
                Eij[now][j] = Eij[vex][j], Eij[j][now] = Eij[j][vex];
        for (int j = 1; j <= tot; ++j)
            if (Root[vex][j]) Root[now][j] = vex;
    }
    CalcSlack(now);
}
void GroupAugment(int x, int y) {
    while (true) {
        int z = Dad[Match[x]];
        SetMatch(x, y);
        if (z == 0) return;
        SetMatch(z, Dad[Pre[z]]);
        x = Dad[Pre[z]]; y = z;
    }
}
bool DealEdge(const Edges &e) {
    int from = Dad[e.from], to = Dad[e.to];
    if (Mark[to] == -1) {
        Pre[to] = e.from;
        Mark[to] = 1; Mark[ Dad[ Match[to] ] ] = 0;
        Slack[to] = Slack[ Dad[ Match[to] ] ] = 0;
        JoinNodeToQueue(Dad[ Match[to] ]);
    } else if (!Mark[to]) {
        int lca = GetLca(from, to);
        if (!lca) {
            GroupAugment(from, to); GroupAugment(to, from);
            for (int i = sizev + 1; i <= tot; ++i)
                if (Dad[i] == i && Expect[i] == 0) BlossomBreak(i);
            return true;
        } else BlossomBuild(from, lca, to);
    }
    return false;
}
void BlossomDelete(int now) {
    for (int v : Leaf[now]) SetDadRec(v, v);
    int root = Root[now][ Eij[now][ Pre[now] ].from ];
    auto it = find(Leaf[now].begin(), Leaf[now].end(), root);
    int place = (int)(it - Leaf[now].begin());
    if (place % 2 == 1) { reverse(Leaf[now].begin() + 1, Leaf[now].end()); place = (int)Leaf[now].size() -
    for (int i = 0; i < place; i += 2) {
        int from = Leaf[now][i], to = Leaf[now][i + 1];
```

```
        Pre[from] = Eij[to][from].from;
        Mark[from] = 1; Mark[to] = 0;
        Slack[from] = 0; CalcSlack(to); JoinNodeToQueue(to);
    }
    Mark[root] = 1; Pre[root] = Pre[now];
    for (int i = place + 1; i < (int)Leaf[now].size(); ++i) {
        int from = Leaf[now][i];
        Mark[from] = -1; CalcSlack(from);
    }
    Dad[now] = 0;
}
bool WorkPhase() {
    while (!Q.empty()) Q.pop();
    for (int i = 1; i <= tot; ++i) { Slack[i] = 0; Mark[i] = -1; }
    for (int i = 1; i <= tot; ++i)
        if (Dad[i] == i && !Match[i]) { Slack[i] = Pre[i] = Mark[i] = 0; JoinNodeToQueue(i); }
    if (Q.empty()) return false;
    while (true) {
        while (!Q.empty()) {
            int from = Q.front(); Q.pop();
            for (int to = 1; to <= sizev; ++to) {
                if (Eij[from][to].data > 0 && Dad[from] != Dad[to]) {
                    if (!CalcEdge(Eij[from][to])) {
                        if (DealEdge(Eij[from][to])) return true;
                    } else if (Mark[Dad[to]] != 1) UpdateSlack(from, Dad[to]);
                }
            }
        }
        ll delta = Inf;
        for (int i = 1; i <= sizev; ++i) if (!Mark[Dad[i]]) delta = min(delta, Expect[i]);
        for (int i = sizev + 1; i <= tot; ++i) if (Dad[i] == i && Mark[i] == 1) delta = min(delta, Expect[i
        for (int i = 1; i <= tot; ++i)
            if (Dad[i] == i && Slack[i])
                if (Mark[i] == -1) delta = min(delta, CalcEdge(Eij[Slack[i]][i]));
                else if (Mark[i] == 0) delta = min(delta, CalcEdge(Eij[Slack[i]][i]) / 2);
        for (int i = 1; i <= sizev; ++i)
            if (Mark[Dad[i]] == 0) Expect[i] -= delta;
            else if (Mark[Dad[i]] == 1) Expect[i] += delta;
        for (int i = sizev + 1; i <= tot; ++i) if (Dad[i] == i)
            if (Mark[i] == 0) Expect[i] += delta * 2;
            else if (Mark[i] == 1) Expect[i] -= delta * 2;
        for (int i = 1; i <= sizev; ++i) if (!Expect[i]) return false;
        for (int i = 1; i <= tot; ++i)
            if (Dad[i] == i && Slack[i] && Dad[Slack[i]] != i && CalcEdge(Eij[Slack[i]][i]) == 0)
                if (DealEdge(Eij[Slack[i]][i])) return true;
        for (int i = sizev + 1; i <= tot; ++i)
            if (Dad[i] == i && Mark[i] == 1 && !Expect[i]) BlossomDelete(i);
    }
    return false;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int m;
    cin >> sizev >> m;
    tot = sizev;
    Eij.assign(MaxV, vector<Edges>(MaxV));
    Root.assign(MaxV, vector<int>(MaxV, 0));
    Leaf.assign(MaxV, vector<int>());
    Match.assign(MaxV, 0); Pre.assign(MaxV, 0); Mark.assign(MaxV, -1); Dad.assign(MaxV, 0);
    Visit.assign(MaxV, 0); Slack.assign(MaxV, 0); Expect.assign(MaxV, 0);
    for (int i = 1; i <= sizev; ++i) for (int j = 1; j <= sizev; ++j) Eij[i][j] = Edges(i, j, 0);
    int maxi = 0;
    for (int i = 0; i < m; ++i) {
        int u, v, w; cin >> u >> v >> w;
        Eij[u][v].data = Eij[v][u].data = w;
        maxi = max(maxi, w);
    }
    for (int i = 1; i <= sizev; ++i) { Match[i] = 0; Dad[i] = i; Leaf[i].clear(); Leaf[i].push_back(i); }
    for (int i = 1; i <= sizev; ++i) for (int j = 1; j <= sizev; ++j) Root[i][j] = (i == j ? i : 0);
    for (int i = 1; i <= sizev; ++i) Expect[i] = maxi;
    while (WorkPhase());
    ll ans = 0;
    for (int i = 1; i <= sizev; ++i) if (Match[i] && Match[i] < i) ans += Eij[i][Match[i]].data;
    cout << ans << '\n';
    for (int i = 1; i <= sizev; ++i) cout << Match[i] << ' ';
    cout << '\n';
    return 0;
}
```

一般图最大独立集

```
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;
const int Maxn=410;
int None[Maxn][Maxn],All[Maxn][Maxn],Some[Maxn][Maxn],Deg[Maxn];
bool Map[Maxn][Maxn];int ans;
inline void FindGraph(int pos,int al,int so,int no)
{
    if(al+so<=ans)return ;
    if(so==0&&no==0){ans=al;return ;}
    int pi=0;
    if(so){
        pi=Some[pos][1];
        for(int i=1;i<=al;i++)All[pos+1][i]=All[pos][i];
    }
    for(register int i=1;i<=so;i++){
        int now=Some[pos][i];
        if(!Map[pi][now])continue;
        int nno=0,nso=0;
        for(register int j=1;j<=so;j++)
            if(!Map[now][Some[pos][j]])Some[pos+1][++nso]=Some[pos][j];
        for(register int j=1;j<=no;j++)
```

```
            if(!Map[now][None[pos][j]])None[pos+1][++nno]=None[pos][j];
        All[pos+1][al+1]=now;
        FindGraph(pos+1,al+1,nso,nno);
        Some[pos][i]=0;None[pos][++no]=now;
    }
}
bool Cmp(int a,int b) {return Deg[a]>Deg[b];}
int n,m;
int main(){
    scanf("%d%d",&n,&m);
    for(register int i=1;i<=n;i++){
        Deg[i]=n-1;
        Some[0][i]=i;
        Map[i][i]=Map[0][i]=Map[i][0]=true;
    }
    for(register int i=1;i<=m;i++){
        int x,y;scanf("%d%d",&x,&y);
        Map[x][y]=Map[y][x]=true;
        Deg[x]--;Deg[y]--;
    }
    sort(Some[0]+1,Some[0]+n+1,Cmp);
    Find_Graph(0,0,n,0);
    printf("%d\n",ans);
    return 0;
}
```

全图割

```
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;

const int MaxN=610;

int Dist[MaxN][MaxN];
int Weight[MaxN],Ord[MaxN];bool Vis[MaxN],Res[MaxN];
int s,t,n,m;

inline int GetMin(int x){
    memset(Vis,false,sizeof(Vis));
    memset(Weight,0,sizeof(Weight));
    Weight[0]=-1;
    for(int i=1;i<=n-x+1;i++){
        int maxi=0;
        for(int j=1;j<=n;j++)
            if(!Vis[j]&&!Res[j]&&Weight[j]>Weight[maxi])
                maxi=j;
        Vis[maxi]=true;Ord[i]=maxi;
        for(int j=1;j<=n;j++)
            if(!Vis[j]&&!Res[j])
                Weight[j]+=Dist[maxi][j];
```

```
    }
    s=Ord[n-x],t=Ord[n-x+1];
    return Weight[t];
}

inline int Solve(){
    int res=0x3f3f3f3f;
    for(int i=1;i<n;i++){
        res=min(res,GetMin(i));
        Res[t]=true;
        for(int j=1;j<=n;j++){
            Dist[s][j]+=Dist[t][j];
            Dist[j][s]+=Dist[j][t];
        }
    }
    return res;
}

int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=m;i++){
        int x,y,c;
        scanf("%d%d%d",&x,&y,&c);
        Dist[x][y]+=c;
        Dist[y][x]+=c;
    }
    printf("%d\n",Solve());
    return 0;
}
```

如果要构造方案，找到使 res 最小时，让那个最后的点所代表的边放在一个连通块中